

STEP-BY-STEP GUIDE TO CREATE AN EF CORE RAZOR PAGES PROJECT AND DEPLOY IT TO AZURE

Step 1: Create a New Razor Pages Project

Step 2: Add EF Core Packages

```
dotnet add package Microsoft.EntityFrameworkCore --version 9.0
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 9.0
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 9.0
dotnet add package Pomelo.EntityFrameworkCore.MySql
```

Step 3: Add a Model (Table Properties)

Step 4: Create DbContext

Step 5: Configure DbContext in Program.cs

SQL Server

```
// Add DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection ")));
```

MySQL

```
// Add DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(
        builder.Configuration.GetConnectionString("DefaultConnection "),
        new MySqlServerVersion(new Version(8, 0, 36)) // your MySQL version
    )
);
```

Step 6: Add Connection String

SQL Server

```
"ConnectionStrings": {
    " DefaultConnection ": "Data Source=localhost;Initial Catalog=STSDB;
User ID=sa;Password=dockerStrongPwd123; TrustServerCertificate=Yes"
}
```

```
"ConnectionStrings": "DefaultConnection": "Data Source=localhost;Initial
Catalog=STSDB;Integrated
Security=True;TrustServerCertificate=True"
}
```

MySQL

```
ConnectionStrings": {  
    " DefaultConnection ":  
    "server=localhost;port=3306;database=schdb;user=root;password=Habeeb@3093;SslMode=N  
one"  
}
```

Step 7: Install EF Core

```
dotnet tool install --global dotnet-ef -version 9.0.0
```

Step 8: Create EF Core Migration

```
dotnet ef migrations add InitialCreate  
dotnet ef database update
```

EF Core (Entity Framework Core) – Definition

Entity Framework Core (EF Core) is a modern **Object–Relational Mapper (ORM)** for .NET.

It allows you to work with a **database using C# classes** instead of writing SQL commands.

You write C# → EF Core converts it to **SQL queries** → sends it to the database.

Why EF Core is Used?

EF Core helps you:

- Create and manage **tables** using C# classes
- Perform CRUD:
 - **Create** (Insert)
 - **Read** (Select)
 - **Update**
 - **Delete**
- Use **Migrations** to create/update database structure
- Avoid writing manual SQL (optional)
- Work with many databases:
 - SQL Server
 - MySQL
 - PostgreSQL
 - SQLite
 - Azure SQL

DbContext – Definition

DbContext is a class in **Entity Framework Core (EF Core)** that manages:

- The **database connection**
- The **tables** in your database (through DbSet<>)
- Running **queries** (SELECT)
- Making **changes** (INSERT, UPDATE, DELETE)
- **Migrations** and database updates

It acts as a **bridge between your C# code and the database**.

Example of EF Core Model

```
public class Students
{
    public int Id { get; set; }
    public string FullName { get; set; }
    public int Age { get; set; }
}
```

Example of EF Core DbContext

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Students> Students { get; set; }
}
```

Classes

Student.cs

```
public class Student
{
    [Key]
    public int ID { get; set; }
    public string Name { get; set; }
    public int age { get; set; }

    public string Country { get; set; }
}
```

AppDbContext.cs

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions options) : base(options)
    {
    }
    public DbSet<Student> Students { get; set; }
}
```